Core Architecture Assumption: RPO = 0 with Manual KRaft Quorum Recovery

The primary objective of this architecture is **data protection**.

The key requirements are:

- **RPO = 0** — no acknowledged Kafka records may be lost.
- **RTO is measured in minutes**, not hours.
- The environment consists of **two data centers only**.
- Kafka topic data is synchronously replicated between both data centers.
- Each partition has **two replicas in DC1 and two replicas in DC2**.
- Loss of the KRaft controller quorum is acceptable during a complete data-center failure, provided that the quorum can be restored through a documented and tested manual Disaster Recovery procedure within the required RTO.

1. The 0.5 DC Is Not Required for Our Primary Objective

A third "0.5 DC" primarily improves **control-plane availability**.

Its main purpose in a 2.5-DC architecture is to provide a tie-breaker for the KRaft controller quorum:

```
DC1          DC2          0.5 DC
C1 C2        C3 C4        C5

Total voters = 5
Majority     = 3
```

After losing either DC1 or DC2, three controllers remain available and the KRaft quorum survives automatically.

This provides an excellent RTO and avoids manual controller quorum recovery.

However, the 0.5 DC does **not provide the fundamental mechanism responsible for RPO = 0**.

RPO protection comes from the Kafka **data replication policy**, not from the location of the KRaft controllers.

Therefore, because our main priority is data durability rather than uninterrupted control-plane availability, we can accept a two-DC controller topology and replace the 0.5-DC requirement with a tested manual quorum recovery procedure.

---

2. Data Plane Design for RPO = 0

The critical architecture is the replication of Kafka partitions between DC1 and DC2.

For example:

```
Replication Factor = 4

DC1:
    Replica R1
    Replica R2

DC2:
    Replica R3
    Replica R4
```

The topic configuration should use:

```
replication.factor=4
min.insync.replicas=3
```

and producers must use:

```
acks=all
```

The important property of this design is:

```
Maximum replicas in one DC = 2
min.insync.replicas        = 3
```

Therefore, an acknowledged write cannot depend exclusively on replicas located in one data center.

For Kafka to acknowledge a write while `min.insync.replicas=3`, at least one in-sync replica must exist in the second data center.

Conceptually:

```
Producer
   |
   | Record X
   v
Leader R1 - DC1
   |
   +---- R2 - DC1
```

```
|
+---- R3 - DC2
|
+---- R4 - DC2
```

Before Kafka returns a successful acknowledgement, the required ISR conditions must be satisfied.

Consequently, an acknowledged record is protected against the complete loss of either individual data center.

---

3. Example: Complete Loss of DC1

Before the failure:

```
DC1                     DC2

R1 - Leader             R3
R2                      R4

ISR = R1,R2,R3,R4
```

A producer writes:

```
Record X
```

and receives a successful acknowledgement.

Then DC1 is completely lost:

```
DC1                     DC2

R1 ❌                    R3 ✅
R2 ❌                    R4 ✅
```

The service may become unavailable because of the controller quorum problem, but the acknowledged record already exists in DC2.

Therefore:

```
Data availability immediately after failure:
possibly unavailable
```

```
Acknowledged data loss:
0
```

This is the distinction between our two main objectives:

```
RPO
===
How much acknowledged data can be lost?

Target: 0


RTO
===
How long does it take to restore Kafka service?

Target: minutes
```

---

4. Control Plane Design

Because only two data centers are available, the KRaft controllers may use an asymmetric topology such as:

```
DC1                     DC2

C1                      C4
C2                      C5
C3

3 controllers           2 controllers
```

Total:

```
5 voters

majority = 3
```

If DC2 fails:

```
DC1:

C1 ✅
C2 ✅
C3 ✅
```

```
3 / 5 available

=> KRaft quorum survives
```

Kafka can continue operating.

The more difficult scenario is the complete loss of DC1:

```
DC1                     DC2

C1 ❌                    C4 ✅
C2 ❌                    C5 ✅
C3 ❌

2 / 5 available

=> KRaft quorum is lost
```

This is an expected and explicitly accepted DR scenario in our architecture.

---

5. Manual Quorum Recovery Is Part of the Architecture

Instead of adding a third 0.5 data center, we accept the following recovery model:

```
DC1 FAILURE
     |
     v
3 controllers lost
     |
     v
KRaft quorum lost
     |
     v
Kafka service unavailable
     |
     v
Execute documented manual
KRaft quorum recovery procedure
     |
     v
Restore controller quorum in DC2
     |
     v
```

```
Validate Kafka metadata
     |
     v
Validate surviving partition replicas
     |
     v
Elect leaders from surviving replicas
     |
     v
Restore producer/consumer traffic
```

The target is therefore:

```
RPO = 0

RTO = several minutes
```

rather than:

```
RPO = 0

RTO ≈ 0
```

provided by a fully operational 2.5-DC architecture.

---

6. Critical DR Principle

During Disaster Recovery, protection of committed data takes precedence over service availability.

We must therefore avoid recovery actions that could cause Kafka to select a replica that does not contain the latest committed data.

In particular, the DR procedure must carefully control:

```
ISR
ELR
High Watermark
leader election
KRaft metadata recovery
min.insync.replicas
```

The recovery procedure must never trade data consistency for faster service restoration without an explicit decision.

For example, lowering:

```
min.insync.replicas=3
```

to:

```
min.insync.replicas=1
```

may restore write availability more quickly, but it changes the durability guarantees of the system.

Such a change must therefore not be an automatic part of the DR procedure.

---

7. Architecture Decision

Our architecture intentionally prioritizes:

```
             PRIORITY

         Data Consistency
               +
         Data Durability
               |
               v
            RPO = 0

               |
               |
               v

         Service Recovery
         RTO = minutes
```

We therefore accept that a complete failure of the data center containing the KRaft controller majority can temporarily make the Kafka cluster unavailable.

We do **not** accept the loss of acknowledged Kafka records.

The architecture consequently uses:

```
DATA PLANE
==========

DC1                     DC2

2 replicas              2 replicas
     \                   /
```

```
      \                /
       RF = 4
       minISR = 3
       acks = all

       Target:
       RPO = 0
```

while the control plane uses:

```
CONTROL PLANE
=============

DC1                    DC2

3 KRaft                2 KRaft
controllers            controllers

        |
        v

Loss of DC containing
controller majority

        |
        v

Manual quorum recovery

        |
        v

Target RTO:
minutes
```

Final Architecture Principle

The absence of a 0.5 DC is an accepted architectural trade-off.

A third lightweight data center would primarily improve controller-quorum availability and reduce RTO.

Because our primary requirement is **RPO = 0**, we instead protect the data plane through synchronous cross-DC replication and accept temporary control-plane unavailability after the loss of the data center containing the controller majority.

The missing automatic KRaft quorum failover will be compensated by a **documented, deterministic, tested manual quorum recovery procedure designed to restore the cluster within several minutes while preserving all acknowledged Kafka data**.